Statystyka - laboratorium 2

1 Wykresy

Zaimportuj plik `births.csv` za pomocą funkcji `read.csv()` i przypisz go do zmiennej o nazwie `births`. Jest to zbiór danych o wagach noworodków urodzonych w pewnym szpitalu. Wykorzystamy go do narysowanie podstawowych wykresów, które poznaliśmy na pierwszym wykładzie:

- `hist(births$BirthWeightGm)` przedstawia histogram masy noworodków,
- `boxplot(births$BirthWeightGm)` przedstawia boxplot masy noworodków.

Zwraca uwagę większa liczba obserwacji odstających w dół, niż w górę. Nie jest to zaskoczenie - dane uwzględniają również narodzone wcześniaki. Staje się to jeszcze bardziej jaskrawe po narysowaniu na jednym wykresie par (waga,tydzień ciąży):

```
plot(births$BirthWeightGm, births$MomAge)
```

Skoro jesteśmy już przy wykresie pudełkowym, możemy go narysować w podziale na pochodzenie matki:

```
boxplot(BirthWeightGm~MomRace, data=births).
```

Na koniec przyjrzymy się dystrybuancie empirycznej. Wykonaj poniższe komendy.

- `ecdf(births$BirthWeightGm)` - zwraca funkcję, więc przypiszmy ją do zmiennej Fhat
- `Fhat <- ecdf(births$BirthWeightGm)`
- `Fhat(3000)` zwróci procent dzieci ważących mniej niż 3kg
- możemy również narysować tę funkcję: `plot(Fhat)`

Zadanie 1. Na podstawie wiadomości zawartych w pliku `h3units.csv` wykonaj polecenia:

(a) Narysuj histogram i dystrybuantę wartości `Health` dla jednostek z "dobrych" zamków, to znaczy **Castle**, **Rampart** i **Tower**.
Wskazówka. Wykorzystaj operator `%in%`. Operator ten pozwala sprawdzić, czy element (lub elementy) po lewej stronie wyrażenia znajduje się w wektorze po prawej stronie wyrażenia, zwracając wektor wartości logicznych.

(b) Narysuj boxploty wartości `Maximum Damage` dla jednostek w podziale na Zamki.

2 Pętle, instrukcje warunkowe i funkcje

Ogólna postać pętli `for` w **R** to:

```
for (x in kolekcjax) {
      ...
}
```

- Część `for (x in kolekcjax)` określa początek pętli. W szczególności:
  - `kolekcjax` to obiekt (np. wektor lub lista), po którym chcemy iterować
  - `x` to zmienna, która przyjmuje kolejno wartości każdego elementu z `kolekcjax`
  - `in` to słowo kluczowe wskazujące na to, że zmienna `x` będzie pobierać kolejne wartości z kolekcji `kolekcjax`
- { ... } to blok kodu, który zostanie wykonany podczas każdej iteracji pętli.

Wykorzystanie pętli `for` zilustrujemy na poniższych przykładach.

Przykład 1. Załóżmy, że chcemy wypisać liczby od 1 do 10. Przykładowy kod wygląda tak:

```
numbers <- c(1:10)
for (num in numbers) {
   print(num)
}
```

Przykład 2. Wyznaczmy dziesiąty wyraz ciągu Fibonacciego.

```
n <- 10
x <- c(0,1)
for(i in 1:n) {
   print(x[1])
   x <- c(x[2], sum(x))
```

}

R pozwala również na definiowanie własnych funkcji przy pomocy składni:

```
nazwa_funkcji <- function(parametr1, parametr2, ...) {
   ...
}
```

`nazwa_funkcji` to po prostu nazwa zmiennej pod którą, zapisany zostanie nagłówek oraz zestaw instrukcji do wykonania. Wynikiem działania funkcji jest jej ostatnie wyrażenie lub argument funkcji `return`, którą można wywołać w dowolnym miejscu ciała definiowanej funkcji.

Definiowane własnej funkcji zilustrujemy na *problemie Collatza*. Ciąg Collatza zdefiniowany jest następująco. Rozpoczynamy od pewnego ustalonego wyrazu x_0. Każdy wyraz powstaje z poprzedniego zgodnie z zasadą: jeśli x_i jest parzyste, to $x_{i+1} = \frac{x_i}{2}$. W przeciwnym przypadku $x_{i+1} = 3x_i + 1$. Hipoteza Collatza mówi, że niezależnie od wyrazu początkowego, w ciągu pojawi się liczba 1, jednak dotychczas nikomu nie udało się znaleźć dowodu. Napiszemy funkcję, która dla danej liczy naturalnej n wygeneruje odpowiadający jej ciąg Collatza (aż do uzyskania liczby 1).

```
collatz <- function(n) {
   seq <- c(n)
   while (n>1) {
      if (n%%2 == 0) {
         n <- n/2
      } else {
         n <- 3*n + 1
      }
      seq <- c(seq, n)
   }
   return(seq)
}

collatz(10)
```

Zadanie 2. Napisz funkcję, która dla zadanej liczby `n` wyznacza liczbę kroków procedury Collatza, która jest potrzebna aby startując od `n` otrzymać jedynkę. Narysuj wykres tej funkcji dla pierwszych 10000 wartości `n`.
Wskazówka. Wykorzystaj funkcję `sapply`, która stosuje daną funkcję do każdego elementu listy lub wektora, zwarcając wynik w postaci wektora. Na przykład, `sapply(X, function(x) x*x)` podniesie każdy element wektora `X` do kwadratu.

3 dplyr

Pakiet `dplyr` służy do prostego i szybkiego przetwarzania danych. Aby zainstalować i załadować pakiet `dplyr` wykonaj poniższe komendy.

```
install.packages("dplyr")
library(dplyr)
```

3.1 Operator %>% - pipe

Standardowo, wartości w **R** przypisujemy za pomocą `<-` lub `=` (możliwe jest także użycie operatora `<-` w drugą stronę, tzn. `->`). W pakiecie `dplyr` mamy dodatkowo operator `%>%` (ang. *pipe operator*) do przetwarzania potokowego, tzn. przekazywania danych z funkcji do funkcji bez pośredniego przypisywania wartości do zmiennej.

3.2 Najważniejsze funkcje

- Wybierz zmienne (kolumny): `select`
- Wybierz obserwacje (wiersze) wg warunku: `filter`
- Ułóż w porządku (posortuj): `arrange`
- Dodaj/zmień istniejącą kolumnę: `mutate`
- Bardziej zaawansowane: `group_by` lub `summarise`

Do zilustrowania operatora `%>%` i pakietu `dplyr` wykorzystamy wbudowany zestaw danych `mtcars`. Załóżmy, że chcemy wybrać te pojazdy ze zbioru `mtcars`, które mają moc powyżej 100 `hp`, a następnie posortować malejąco względem `mpg`.

```
mtcars %>%
   filter(hp>100) %>% # wybranie odpowiednich rekordów
   arrange(desc(mpg)) %>% # sortowanie
   head() # wyświetlenie kilku pierwszych rekordów
```

Wynik działania poprzedzającego polecenie staje się pierwszym argumentem kolejengo polecenia. Stąd też `head` jest niejako bez jawnego argumentu.

Rozważmy teraz dane `births`. Policzymy najpierw średnią wagę noworodka w podziale na rasę matki:

```
births %>%
   group_by(RaceMom) %>%
   summarise(avg = mean(BirthWeightGm))
```

Dodamy teraz nową kolumnę z masą noworodka wyrażoną w kiogramach:

```
births %>% mutate(massKg = BirthWeightGm/1000)
```

Zadanie 3. Co będzie wynikiem poniższej komendy?

```
births %>%
   filter(BirthWeightGm>3000) %>%
   select(c("Weeks", "RaceMom")) %>%
   distinct()
```

4 ggplot2

Biblioteka `ggplot2` służy do przygotowywania grafiki naukowej w **R**. Przed użyciem bibliotekę należy zaintalować komendą `install.packages("ggplot2")`, a następnie załadować używając komendy `library(ggplot2)`.

Sposób przygotowywania wykresów z użyciem `ggplot2` jest dość nietypowy, ale stosunkowo intuicyjny. Na początku tworzymy pusty wykres komendą `ggplot()`. Na tym etapie możemy od razu określić, z jakich danych chcemy korzystać, pisząc np. `ggplot(mtcars)`. Dane muszą być typu **data frame**. Jest to (niemalże) jedyny typ danych obsługiwany przez tę bibliotekę. Częstym źródłem błędów przy obsłudze biblioteki `ggplot2` jest próba stworzenia wykresu z wektorów liczb zmiennoprzecinkowych.

Na pusty wykres nakładamy kolejne warstwy, takie jak wykres punktowy, liniowy, histogram itd. Każdą taką warstwę tworzymy osobną komendą, zaczynającą się na ogół od `geom_` lub od `stat_`. Na przykład, `geom_point()` to warstwa zawierająca wykres punktowy. Wszystkie geometrie (czyli typy wykresów) są opisane w dokumentacji pakietu `ggplot2` tutaj. Poszczególne elementy łączymy ze sobą plusami, na przykład `ggplot() + geom_point()`. Na końcu musimy jeszcze określić, co konkretnie chcemy wykreślić na wykresie. W `ggplot2` robimy to za pomocą tzw. *estetyk* (ang. *aesthetics*), które określają, w jaki sposób dane mają być przekształcone na własności wizualne na wykresie. Estetyki określają m.in. położenie na osi x i na osi y, kolor, rozmiar i wypełnienie punktów. Możemy określić je globalnie, w funkcji `ggplot()`, lub dla każdej warstwy z osobna.

Przykładowa komenda:

```
ggplot(births) +
   geom_point(aes(x=Weeks, y=BirthWeightGm, col=as.factor(Smoke)))
```

- `ggplot(births)` - pusty wykres, wszystkie warstwy pracowałyby ze zbiorem `births`
- `geom_point()` - będzie wykres punktowy
- `aes(x=Weeks, y=BirthWeightGm, col=as.factor(Smoke))` - estetyka dla wykresu punktowego. x i y to dane, które umieścimy. Punkty zostaną pokolorowane na podstawie wartości zmiennej `Smoke`
- Wiemy z jakiego zbioru pochodzą zmienne - zostało to przekazane w `ggplot()`.

Rozmiar punktów określamy podając argument `size`, na przykład:

```
ggplot(births) +
   geom_point(size=.5, aes(x=Weeks, y=BirthWeightGm, col=as.factor(Smoke)))
```

Żeby zmodyfikować ogólny wygląd wykresu, możemy dodać tzw. *temat* (ang. *theme*). Do grafiki naukowej szczególnie nadaje się `theme_minimal()`:

```
ggplot(births, aes(x=Weeks, y=BirthWeightGm) ) +
   geom_point(size=.5, aes(col=as.factor(Smoke))) +
   theme_minimal() +
   xlab("Tydzień")
```

Przyjrzyj się wykresom generowanym przez poniższe komendy:

```
birth_plot <- ggplot(births) + theme_minimal()

birth_plot + aes(group=RaceMom, x=RaceMom, y=BirthWeightGm) + geom_violin()

birth_plot + aes(x=RaceMom) + geom_bar() +
   geom_violin(aes(group=RaceMom,y=BirthWeightGm))

birth_plot + aes(x=Weeks, y=BirthWeightGm) + geom_point(size=.5) +
   geom_smooth(method="lm") + facet_grid(vars(HispMom), vars(Smoke))
```

`facet_grid()` to funkcja służąca do tworzenia macierzy paneli (siatki wykresów), która dzieli dane według dwóch zmiennych kategorycznych – jednej dla wierszy, drugiej dla kolumn. Pozwala to na porównywanie wykresów dla wszystkich kombinacji tych zmiennych, co jest przydatne do wizualizacji powiązań między dwoma grupami danych w tej samej siatce.

Zadanie 4. Funkcja `rnorm()` służy do losowania obserwacji z rozkładu normalnego. Wywołanie komendy `help(rnorm)` wyświetli jej dokumentację, wraz z opisem kilku innych funkcji związanych z rozkładem normalnym. Celem tego zadania jest porównanie gęstości rozkładu normalnego z histogramem wylosowanych wartości.

- Utwórz wektor `X` zawierający 200 obserwacji wylosowanych z rozkładu normalnego z parametrami `mean=4` oraz `sd=2`.
- Przekształć wylosowany wektor w ramkę danych: `X <- data.frame(X)`.
- Utwórz histogram wylosowanych wartości korzystając z biblioteki `ggplot2` i geometrii `geom_histogram`. Dokumentację i przykłady zastosowania tej geometrii wyszukaj w internecie. Co znajduje się na osi OY utworzonego wykresu?
- Spraw, żeby na osi OY znalazła się częstość występowania danej wartości. Można to zrobić wykorzystując specjalną estetykę: `aes(y=..density..)`.

- Utwórz wektor zmiennoprzecinkowy `x`, zawierający ciąg liczb od -5 do 15 z odstępem 0.2 (tak, żeby długość wektora `x` wynosiła 101). Możesz wykorzystać funkcję `seq`.
- Utwórz wektor `y`, zawierający wartość gęstości rozkładu normalnego z parametrami `mean=4`, `sd=2` w punktach wektora `x`: `y <- dnorm(x, 4, 2)`. Większość funkcji w **R**, w tym `dnorm`, przyjmuje jako argument całe wektory i zwraca wynik obliczony dla każdego elementu wektora.
- Połącz wektory `x` i `y` w jedną ramkę danych o nazwie `d`.
- Do histogramu utworzonego w poprzednich punktach dodaj warstwę zawierającą wykres liniowy obrazujący obliczoną gęstość. Pamiętaj, że dane do stworzenia nowej warstwy możesz podać bezpośrednio w geometrii: `geom_line(aes(...), data=d)`.

Wywołaj swój kod kilkukrotnie i zobacz, w jaki sposób zmienia się histogram. Wypróbuj również kilka wartości parametrów `mean` oraz `sd`.

Zadania dodatkowe

1. Plik `starwars.csv` zawiera informacje o postaciach występujących w Gwiezdnych Wojnach. Przy pomocy funkcji zawartych w pakietach `dplyr` i `ggplot2` odpowiedz na następujące pytania/wykonaj następujące polecenia:
   (a) jakiego wzrostu był **Darth Maul**?
   (b) kto był najcięższą postacią, która występowała w **The Empire Strikes Back**? Czy jest to najcięższa ze wszystkich postaci?
   (c) BMI to $\frac{wzrost}{masa^2}$. Przedstaw minimalny, średni i maksymalny BMI w podziale na planety.
   (d) Wyświetl postacie, które wystąpiły w co najmniej trzech filmach.
   (e) Narysuj wykres zależności masy od wieku. Kolorem oznacz płcie. Zaznacz na wykresie najlepiej dopasowaną prostą. Wykonaj ten rysunek w podziale na płcie.
2. Rozważmy uproszczoną grę w *chińczyka*: poruszamy się pionkiem po polach ponumerowanych od 1 do 40. Gra dwóch graczy: Xawery i Yarosław. Przy każdym ruchu rzucamy kostką. Jeżeli wypadło 6 oczek, to stoimy w miejscu. W przeciwnym przypadku poruszamy się do przodu o tyle, ile wypadło oczek. Napisz kod, który symuluje taką rozgrywkę - jako rezultat powinniśmy dostać wektory `x` i `y` położeń graczy po kolejnych rundach.